后训练学习计划 • 第三周（9/21 - 9/27）

> 本周定位：**Phase 1 第二周——把 Transformer Block 从骨架啃到 nanoGPT 源码**。主题是「(T,C) 的玩具 attention 升级为 (B,T,C) 多头因果注意力 + 完整 Block」，然后逐行精读 nanoGPT 的 `CausalSelfAttention / MLP / Block / GPT`。上周你会算 loss 了，这周你要知道这个 loss 在优化**什么结构**。
> 用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，`> 注` 块 = 注释（可跳过，不影响主线）。本周闭合 Week 1 自测第 4 题（画 Transformer decoder block）——那个洞被明确推到了这里；本周不发博客，但「前向形状表」是 Week 5 博客的核心素材，按素材归档。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	(T,C) → (B,T,C) 多头因果注意力（切分/转置/合并）	`block.py` 的 attention 部分 + 7 个单测全绿	6.5h
Day 2 周二	位置信息：Embedding 查表、可学习绝对位置 vs RoPE	置换等变实验 + RoPE 相对位置不变性 assert 通过	6h
Day 3 周三	LayerNorm vs RMSNorm、FFN/4C/GELU/SwiGLU、Block 组装	两种 norm 各与 torch 对拍 OK + `Block` 前向形状不变	6.5h
Day 4 周四	nanoGPT `model.py` 逐行精读	grep 审计表（≥15 条断言带行号）+ 细读笔记 30 条	6.5h
Day 5 周五	与论文差异清单、MHA/MQA/GQA/KV cache、SDPA 对拍、上卡 1000 步	差异表 ≥6 条有据 + `SDPA CROSSCHECK OK` + hook 形状全对	6h
Day 6 周六	白板默写 block + 形状表定稿 + 闭卷自测 4 题 + repo push	形状表 12 行 + 自测卷批改 + GitHub 链接	6h
Day 7 周日	补漏 + 闭卷复测 + 总验收 + Week 4 预习	验收 ≥6/7 项通过	4.5h

> 注：本周**卡时上限 2h、预估花费 ¥3-5**。唯一用途是 Day 5 任务 4：跑 nanoGPT 的 shakespeare_char 配置 1000 步，看一条真实 loss 曲线 + 用 forward hook 验证你自己的形状表。**不要顺手把 Week 5 的 M1 训练提前跑完**——那是下周的活，今天多跑 8 小时，下周就没钱调 lr 了（P1 全程 30h 上限：本周 2h + 下周 12h + Week 4 的 8h = 22h，留 8h 余量）。
> 注：本周最大风险是**在 `view/transpose/reshape` 上耗光晚上**——所有维度问题用同一个办法解决：`print(t.shape)` 或 `python -c "import torch; ..."` 现推，形状能打印出来就永远不该靠记忆。第二风险是**读源码读成小说**：Day 4 全程带 grep 审计表读，每 10 分钟必须落一条带行号的结论。本周**故意不含**训练工程（DataLoader/AMP/累积/checkpoint 全在 Week 4）、不含真实模型训练（Week 5）、不含 MoE/MLA 的数学细节（能讲清概念即可，Week 14 八股再补）。

二、开始前：环境、仓库与上周欠账（约 25 分钟，前一晚做完）

1. 环境仍是 `E:/study/posttrain/.venv`（**别在 conda base 里跑 torch**，会 `OMP: Error #15: Access is violated` / `libiomp5md.dll already initialized`；临时绕过 `export KMP_DUPLICATE_LIB_OK=TRUE`，正解是回

venv）。`torch` 必须 ≥ 2.0（Day 5 要用 `F.scaled_dot_product_attention`），低于就 `pip install -U torch`；装不动就把 Day 5 任务 3 改成「和自己写的 einsum 版对拍」，别在装 torch 上耗晚上：

```
source E:/study/posttrain/.venv/Scripts/activate
pip install "numpy>=1.24" matplotlib pytest -i https://pypi.tuna.tsinghua.edu.cn/simple
python -c "import torch;print(torch.__version__, torch.cuda.is_available())"   # 必须 >= 2.0
mkdir -p E:/study/posttrain/week03/{day1,day2,day3,day4,day5,day6}
ls E:/study/posttrain/week02/makemore/vocab.py       # Day 2 降级路线与 Day 5 字符数据要用
ls E:/study/posttrain/day5/attention.py              # Week 1 那份 (T,C) 单头实现,今天升级它
```

完成判据：上面 bash 块 6 行全部成功、`torch.__version__` ≥ 2.0（否则已记降级方案）、`week03/` 六个子目录存在；并已打开 `week02/week02-gaps.md`（Day 1 任务 1 要按它认领）。

Day 1 • 周一：把 attention 升级到 batch + 多头——`view` 和 `transpose` 是今天全部的难点

今日总览（6.5h）：上周（Week 1）你写过 `(T,C)` 的单头 attention，那版**没有 batch、没有多头、没有位置编码**。今天三样补齐头两样。晚上你要拿到：`week03/block.py` 里一个 `(B,T,C) → (B,T,C)` 的多头因果注意力函数，每一步都有形状注释，且 7 个单测全绿（包括「改最后一个 token 不得改变前面任何输出」这条真正管用的因果性测试）。

任务 1：开工对账——从 `week02-gaps.md` 里清哪几条（约 20 分钟）

打开 `week02/week02-gaps.md`，把条目按下面认领，抄进 `week03/day1/today.md`：

1. **多头 / 带 batch 的 attention、Block 整体结构、位置编码怎么进网络** → 本周 Day 1（多头）、Day 2（位置）、Day 3（Block）。这是 Week 1 自测第 4 题那个洞的正主，做完 Day 6 的白板默写才算闭合。
2. **BatchNorm / Dropout 的行为差异**（Week 2 明确没跟 `bigger.py`）→ 本周 Day 3 任务 4 顺带处理 dropout（Block 里要不要放、放哪），BN 推到 **Week 4 Day 2**（`model.train()/eval()` 那节）。
3. **采样 / temperature / top-k 只在 toy 上试过** → **Week 5** 的采样对照小表，本周不管。
4. **优化器状态的显存代价、lr 调度在真训练里怎么配** → **Week 4 Day 3/Day 5**，本周不管。
5. **长上下文与真实分词（BPE）** → **Week 5 + Week 6**；本周 Day 5 上卡时第一次看到 nanoGPT 的 char/BPE 数据脚本，只看不做。
6. Week 2 的**遗留未勾项**（若还有 `[ ]`）：能今天清的就清，不能的写进本周 gaps，**不许默默消失**。

完成判据：`today.md` 里 ≥6 行认领，其中第 1/2 条写明本周哪天，第 3/4/5 条写明哪周；若第 6 条存在则必须给出「今天做掉」或「挪到 Week N」的明确结论。

> 注：这张「认领—归派」表是这套计划最容易被跳过、收益却最高的一步。到 Week 14 整理失败案例清单时，你要靠它回溯「这个坑我第几周就见过、当时怎么处理的」——面试官问「你踩过什么坑」，最有说服力的回答是带时间线的。

任务 2：多头的数学与「切分」到底在切什么（约 60 分钟）

1. 拿 `C=8, h=2 → hs=4` 这个小例子手推：`Wq` 是 `8×8`，`x@Wq` 的第 `j` 列属于哪个头？答案是**前 4 列是第 0 个头**，因为 nanoGPT/`nn.Linear` 都是「按输出列连续切分」。把这条结论写成 numpy 断言（下一步的实现必须满足它）。
2. 说清「多头到底买到了什么」：`h` 个头各自在 `hs=C/h` 维子空间里做注意力，再拼接过 `Wo` ——**等价于 `h` 次独立的低维注意力**，不是「算了 h 遍同一个大注意力」。
3. 手算参数量：一个 attention 层 `4C²`（QKOV）+ Block 里 FFN `8C²`（`C→4C→C`）= `12C²`，先记着，Day 3 任务 5 要用 `numel()` 对拍。
4. NOTES 写两条：为什么 `hs` 而不是 `C` 进 `√` 缩放（每个头的点积只在 `hs` 维上做，方差按 `hs` 计）；为什么 `h` 增大而 `C` 不变时**参数量几乎不变**（还是 `4C²`），只是每个头变窄。

完成判据：NOTES 两条结论齐全，且你能不看资料口头回答「h=8, C=512 时 head_dim 是多少、attention 矩阵形状是什么」（`(B,8,T,T)`）。

任务 3：实现 `(B,T,C)` 多头因果注意力（约 160 分钟）

新建 `week03/block.py`，先写 attention 部分（`causal` **开关今天就加上，Day 2 的置换实验要用**）。**下面这份是参考实现，先把 6 处 `# TODO` 的注释自己写出来再对照**（注释里要出现形状）：

```
import math
import torch
def causal_multihead_attention(x, Wq, Wk, Wv, Wo, n_head, causal=True):
    """x:(B,T,C)  Wq/Wk/Wv:(C,c)（c=注意力输出维,通常 c=C）->  (B,T,c), 权重 (B,h,T,T)"""
    B, T, C = x.shape
    assert Wq.shape[1] % n_head == 0, "注意力输出维必须能被头数整除"
    hs = Wq.shape[1] // n_head                          # 每个头的维度 = 输出维/头数
    q = (x @ Wq).view(B, T, n_head, hs).transpose(1, 2)   # TODO: 为什么必须 transpose(1,2)
    k = (x @ Wk).view(B, T, n_head, hs).transpose(1, 2)
    v = (x @ Wv).view(B, T, n_head, hs).transpose(1, 2)   # 三个都是 (B,h,T,hs)
    att = q @ k.transpose(-2, -1) / math.sqrt(hs)         # TODO: 为什么是 transpose(-2,-1)
    if causal:                                            # TODO: mask 为什么必须在 softmax 之前
        att = att + torch.triu(torch.full((T, T), float("-inf"), dtype=x.dtype), diagonal=1)
    att = torch.softmax(att, dim=-1)                      # TODO: 归一化在哪一维,为什么是 -1
    y = (att @ v).transpose(1, 2).reshape(B, T, -1)       # TODO: 为什么这里要 reshape 而不是 view
    return y @ Wo, att
```

注意 `hs` 是从 `Wq` 的输出列数算的，不是从 `C` 算的——`nn.Linear` 只管输出维，头切分切的是输出列；写成 `C // n_head` 在 `c_attn` 输出 `3C` 的真实现场立刻错位（nanoGPT 里就是这么算的，Day 4 自己对行号）。`reshape(B, T, -1)` 同理，别把 `C` 写死。

再写第二个实现 `causal_multihead_einsum`（**不 transpose**，靠下标直接表达：`att = torch.einsum("bthd,bshd->bhts", q, k)`，`y = torch.einsum("bhts,bshd->bthd", att, v)`，其中 q/k/v 都先 reshape 成 `(B,T,h,hs)`），两者结果必须完全一致——这一步是防自己「只会抄 nanoGPT 的转置」。

完成判据：两个实现都跑通；`y.shape == (B,T,C)`；把 `att.shape` 打印出来必须是 `(B,h,T,T)`；打印 `ATTENTION BUILT OK`。

> 注：`transpose(1,2)` 之后张量不再连续，`view` 会直接报 `RuntimeError: view size is not compatible with input tensor's size and stride` ——**这就是注释里那句 `reshape` 的原因**（reshape 会自动拷贝，view 不会）。另一个高频静默错误：`view(B, T, n_head, hs)` 写成 `view(B, n_head, T, hs)`，前向不报错、梯度也不报错，loss 只是降不下去。**转置的顺序永远靠打印 shape 来确认，不靠脑子。**

任务 4：单测——因果性要测输出，不是测权重（约 120 分钟）

新建 `week03/day1/test_block.py`（pytest 或文件末尾那个 `for fn in [...]` 循环都行）。`block.py` 在上一级目录，所以在 `week03/` 下跑 `PYTHONPATH=. python day1/test_block.py`（PowerShell：`$env:PYTHONPATH="."; python day1/test_block.py`）。七个测试：

```
import torch
from block import causal_multihead_attention, causal_multihead_einsum
torch.manual_seed(1234)                    # 写死:失败时别人能复现你的失败
B, T, C, H = 3, 7, 8, 2
W = lambda: torch.randn(C, C, dtype=torch.float64) * 0.5
def make():
    return W(), W(), W(), W()
def rx():                                   # 每次现取,免得测试之间共享 x
    return torch.randn(B, T, C, dtype=torch.float64)
def test_shape():
    y, att = causal_multihead_attention(rx(), *make(), H)
    assert y.shape == (B, T, C) and att.shape == (B, H, T, T)
def test_rows_sum_one():
    _, att = causal_multihead_attention(rx(), *make(), H)
    assert torch.allclose(att.sum(-1), torch.ones(B, H, T, dtype=torch.float64), atol=1e-10)
def test_upper_triangle_zero():
    _, att = causal_multihead_attention(rx(), *make(), H)
    assert torch.allclose(torch.triu(att, diagonal=1), torch.zeros_like(att))
def test_causal_output_unchanged():         # 真正管用的那条
    x = torch.randn(B, T, C, dtype=torch.float64)
    wq, wk, wv, wo = make()
    y1, _ = causal_multihead_attention(x, wq, wk, wv, wo, H)
    x2 = x.clone(); x2[:, -1] = torch.randn(B, C, dtype=torch.float64)
    y2, _ = causal_multihead_attention(x2, wq, wk, wv, wo, H)
    assert torch.allclose(y1[:, :-1], y2[:, :-1], atol=1e-12)
def test_first_row_self_attention():
    _, att = causal_multihead_attention(rx(), *make(), H)
    assert torch.allclose(att[:, :, 0, 0], torch.ones(B, H, dtype=torch.float64))
def test_head_split_equivalence():          # 呼应任务 2 第 1 条
    x = rx()
    wq, wk, wv, wo = make()
    hs = C // H
    _, att_full = causal_multihead_attention(x, wq, wk, wv, wo, H)
    for h in range(H):                      # 第 h 个头:只取 Wq/Wk/Wv 的对应列段,当单头算
        sl = slice(h * hs, (h + 1) * hs)
        _, att_h = causal_multihead_attention(x, wq[:, sl], wk[:, sl], wv[:, sl],
                                              torch.eye(hs, dtype=torch.float64), 1)
        assert torch.allclose(att_h[:, 0], att_full[:, h], atol=1e-12)   # 都是 (B,T,T)
def test_two_impls_agree():
    x = rx()
    a = make()
    y1, t1 = causal_multihead_attention(x, *a, H)
    y2, t2 = causal_multihead_einsum(x, *a, H)
    assert torch.allclose(y1, y2, atol=1e-12) and torch.allclose(t1, t2, atol=1e-12)
if __name__ == "__main__":
    for fn in [v for k, v in sorted(globals().items()) if k.startswith("test_")]:
        fn(); print(fn.__name__, "PASS")
    print("ALL BLOCK TESTS PASS")
```

`test_head_split_equivalence` 的含义要说清楚：**把 `Wq` 按输出列切成 `h` 段，每段单独当单头算，得到的注意力矩阵必须和多头版里第 `h` 个头逐元素相同**。这条一挂，就说明你的切头方式与 nanoGPT/`nn.Linear` 的约定不一致（写错不报错、只是静默学不好，是最阴的一类 bug）。它之所以能跑，靠的就是任务 3 里那句 `hs =`

`Wq.shape[1] // n_head` ——如果你的实现非要从 `x.shape[-1]` 反推 `hs`，这个测试会先在 `view` 上炸掉，**那就对了**。

完成判据：打印 7 个 `PASS` + `ALL BLOCK TESTS PASS`；`test_causal_output_unchanged` 必须存在（只有它才真正验证「未来影响不到过去」，前三个只查了权重矩阵）。

> 注：`test_first_row_self_attention` 断言 `att[:,:,0,0] == 1` ——第一个 token 只能看自己，softmax 归一化后必然全是它。这条不是「观察」，是**数学必然后果**，它挂了说明 mask 或归一化轴错了。反过来，**别去断言「注意力会 attend 到邻近 token」**：随机初始化下没有局部性可言（Week 1 的可视化任务里就埋过这个坑），写了这种断言你会以为是代码错了。

任务 5：收工笔记（约 30 分钟）

`week03/day1/shapes.md` 开一张空表，把今天的 `x / q / k / v / att / y` 六个张量的形状填进去（这就是 Day 6 那张「前向形状表」的第一块砖）。NOTES 写：今天最痛的一次 `view/reshape/transpose` 报错，原文错误消息 + 修法。

完成判据：形状表 6 行齐全 + `ALL BLOCK TESTS PASS` 的终端截图或粘贴 + NOTES ≥2 条含报错原文。

今日验收（6.5h）：两个实现一致 + 7 测全绿 + 形状表起头 + gaps 认领 6 行。

Day 2 • 周二：位置信息——没有位置编码的 attention 是个瞎子

今日总览（6h）：昨天那版 attention 对 token 顺序是**置换等变**的：打乱输入，输出只是同样被打乱，值一点不变。这意味着「顺序」必须由别的东西注入。今天先用实验把这件事钉死，再写可学习绝对位置嵌入（GPT-2/nanoGPT 的做法）与 RoPE（LLaMA 的做法），并证明 RoPE 的招牌性质：`q_m·k_n` 只依赖 `m-n`。

任务 1：《Let's build GPT》开场 + 数据就位（约 55 分钟）

1. 看 Karpathy 系列第三讲 **「Let's build GPT: from scratch, in code, spelled out」** 的前 40 分钟：tokenization（他用的极小莎士比亚切片，约 4 万字符、字符表约 65 个——**以他 print 出来的为准，别背我的数**）、`wte/wpe`、以及为什么 embedding 要乘 `√n_embd`。
2. 本地数据两条路（走通一条就停）：
   - 首选：`git clone https://github.com/karpathy/nanoGPT E:/study/posttrain/nanoGPT`（Day 4 也要用）后跑 `python data/shakespeare_char/prepare.py` ——**只跑 prepare，不训练**，训练留到 Day 5。
   - 降级：复用 Week 2 的 `vocab.py` + 一小段莎士比亚纯文本（gutenberg），自己造 `(B,T,C)` 输入张量。
3. 把打印出来的 `vocab_size` / 数据长度两个真实数字记进 NOTES。

完成判据：手上有一份能 `random` 出 `(B=2, T=8)` 批次的字符级数据（或 nanoGPT 的 `get_batch` 跑通），打印两个 batch 的 `xs.shape`、`ys.shape` 与解码回来的 200 字符原文（能看出是莎士比亚、且 `xs` 与 `ys` 恰好错一

位）。

任务 2：置换等变实验——证明 attention 自己看不见顺序（约 60 分钟）

`week03/day2/permutation_test.py`：直接用昨天 `block.py` 里的 `causal=False` 分支：

```
import torch
from block import causal_multihead_attention   # 用你的版本,支持 causal=False
torch.manual_seed(7)
B, T, C, H = 2, 6, 8, 2
x = torch.randn(B, T, C, dtype=torch.float64)
W = [torch.randn(C, C, dtype=torch.float64) * 0.5 for _ in range(4)]
perm = torch.randperm(T, generator=torch.Generator().manual_seed(8))
y1, _ = causal_multihead_attention(x, *W, H, causal=False)
y2, _ = causal_multihead_attention(x[:, perm], *W, H, causal=False)
print("等变? ", torch.allclose(y1[:, perm], y2, atol=1e-12))         # True
print("顺序敏感? ", torch.allclose(y1, y2, atol=1e-12))              # False(几乎必然)
```

完成判据：两行分别打印 `True` 与 `False`；再加一段：同一实验**打开 causal mask** 后，等变性**不再成立**（打印 `False`），并在 NOTES 解释为什么（mask 是按绝对位置下标写的，置换输入等于同时置换了「谁被允许看谁」）。这一条是很多人第一次搞错的地方。

> 注：这个实验的价值在于它把「self-attention 是集合算子（permutation invariant/equivariant）」这句八股变成了你自己跑出来的结论。面试问「attention 为什么需要位置编码」，90% 的人答「因为 attention 是置换不变的」，能补一句「我把 mask 关掉做过等变性测试，打开 mask 就不成立了，因为 mask 编码的是绝对位置」的人不到 10%。

任务 3：Embedding 查表、`√n_embd` 缩放与 one-hot 等价（约 70 分钟）

1. 写 `week03/day2/embed.py`：`wte = nn.Embedding(V, C)`，输入 `idx (B,T)` → 输出 `(B,T,C)`；把 `wte(idx)` 与 `onehot(idx) @ wte.weight` 在 float64 下对拍（`atol=1e-12`），证明上周 NOTES 里那句「Embedding 就是查表」。
2. `wpe` 用 `nn.Embedding(block_size, C)`，前向 `x = wte(idx) + wpe(pos)`（要求 `T ≤ block_size`）；再实现 `wte.weight.data *= math.sqrt(C)`（GPT-2 那套），解释清楚**为什么**：`wte` 与 `wpe` 都是 `N(0,1)` 量级，相加后进入 LayerNorm 会被归一化掉，把 `wte` 放大 `√C` 倍（C=256 时是 16 倍）是让「词的身份」在被 norm 压平之后仍占主导。写成 3 行 NOTES，别抄我的表述。

完成判据：one-hot 对拍打印 `EMB EQUIV OK`（diff <1e-12）；`x = wte(idx) + wpe(pos)` 前向打印形状 `(B,T,C)`，且把 `wte` 换成 `wte/math.sqrt(C)` 后 `x` 的均值绝对值明显变小（打印两个数对比）；NOTES 有「为什么乘 √n_embd」的 3 行，必须出现「LayerNorm 会把尺度归掉」这层意思。

任务 4：RoPE 的 numpy/torch 版与「相对位置不变性」证明（约 120 分钟）

新建 `week03/day2/rope.py`：

```
import torch
def rope(x, start=0, base=10000.0):
    """x: (B,h,T,hs), hs 为偶数。按相邻两维 (2m,2m+1) 做二维旋转,角度 = pos * inv_freq"""
    B, h, T, hs = x.shape
    assert hs % 2 == 0, "head_size 必须是偶数"
    pos = torch.arange(start, start + T, dtype=torch.float64)
    inv_freq = 1.0 / (base ** (torch.arange(0, hs, 2, dtype=torch.float64) / hs))
    ang = pos[:, None] * inv_freq[None, :]                # (T, hs/2)
    cos, sin = ang.cos()[None, None], ang.sin()[None, None]
    x1, x2 = x[..., 0::2], x[..., 1::2]
    out = torch.empty_like(x)
    out[..., 0::2] = x1 * cos - x2 * sin
    out[..., 1::2] = x1 * sin + x2 * cos
    return out
if __name__ == "__main__":
    torch.manual_seed(0)
    q = torch.randn(1, 2, 5, 8, dtype=torch.float64)
    k = torch.randn(1, 2, 5, 8, dtype=torch.float64)
    a = rope(q) @ rope(k).transpose(-2, -1)               # 绝对位置 0..4
    b = rope(q, start=3) @ rope(k, start=3).transpose(-2, -1)   # 整段平移 3
    print("平移后分数是否不变:", torch.allclose(a, b, atol=1e-10))
    assert torch.allclose(a, b, atol=1e-10)
    print("ROPE RELATIVE OK")
```

再补两个观察（写进 NOTES，各配一个数）：① `inv_freq` 随维度指数衰减 → 低维转得快、高维转得慢，这正是「多尺度相对位置」的来源；② RoPE 不需要 `wpe` 表，所以序列长度超过训练长度时**不用改结构**（这是外推问题的其中一半，另一半是 NTK/YaRN，不在本周范围，知道名词即可）。

完成判据：打印 `ROPE RELATIVE OK` + `True`；两个观察各有具体数（比如第 0 维与第 6 维的 `inv_freq` 各是多少）。

> 注：这里用的是**相邻配对**（`0::2 / 1::2`）的原始 RoPE 写法。HuggingFace 里 LLaMA 的实现是 `rotate_half`（前后半切分），**两者只差一个维度置换**，读源码时容易以为看到两种不同算法——今天写一遍，读源码时就能认出这是同一件事。这类「同一个东西两个名字」的识别能力，比多写一个变体值钱。

任务 5：可学习绝对位置 vs RoPE 对照小表 + 收工（约 55 分钟）

`week03/day2/pos_notes.md` 一张 5 行表（顶层表格）：

维度	可学习绝对位置（GPT-2 / nanoGPT）	RoPE（LLaMA 系）
参数放在哪	`wpe` 一张 `block_size × C` 的表	无参数，公式作用在 q/k 上
长度上限	硬上限 = `block_size`，超出要改结构	结构无上限，但外推质量会掉
作用位置	加到输入嵌入上	只在 q/k 上，不碰 v
相对 vs 绝对	绝对位置，靠模型自己学出相对关系	直接给出相对位置（`m-n`）
我要实测的东西	今天不训，Week 5 用 nanoGPT 时会撞 `block_size`	平移不变性今天已 assert

完成判据：表 5 行齐全且「作用位置」那一行的 `v` 说对了（**RoPE 不旋转 V**，只旋转 Q/K——因为旋转是正交变换，会破坏 V 作为「内容」的可加性；能讲成自己的话即可）。**这一行 Week 3 自测会考**。

今日验收（6h）：置换等变两行 True/False + `EMB EQUIV OK` + `ROPE RELATIVE OK` + 位置对照表 5 行。

Day 3 • 周三：norm、FFN 与 Block——今天结束时要有一个完整的、形状不变的 Block

今日总览（6.5h）：Block = 因果多头注意力 + FFN，各自前面一个 norm、后面一条残差。今天你要：① 手写 LayerNorm 与 RMSNorm 并各与 torch 对拍；② 看清 FFN 的 `C→4C→C` 与 GELU/SwiGLU；③ 组装 Pre-LN Block，做到**输入输出形状严格相同**，并让参数量对上 `12C²`。今天之后，`week03/block.py` 是一个可以喂给 Week 4 训练循环的模块。

任务 1：LayerNorm 手推 + numpy 版（约 100 分钟）

1. 写出定义：对**最后一维**（特征维 C）求均值与方差、归一化、再乘 `γ` 加 `β`。三个问题必须能答：
   - 为什么在 **C** 上归一化而不是 **T**（LayerNorm 与 BatchNorm 的根本差别；序列长度可变、batch 内各样本互不影响，这是 Transformer 选 LN 的原因）。
   - 方差是除以 C 还是 C−1（torch 用的是**有偏**的除以 C，别自己"更正"它，否则对拍差 `C/(C-1)` 这个因子）。
   - `eps` 放根号内还是根号外（torch 是 `(x-μ)/√(σ²+eps)`，**在根号内**）。
2. numpy 实现 + `elementwise_affine` 支持；用 `x.mean(-1)`、`x.var(-1)`，别用 `np.std` 凑（`std` 默认 ddof=0，恰好一致，但要写明）。
3. 观察并写进 NOTES：LN 之后每个位置的输出**均值≈0、方差≈1**（打印 `y.mean(-1)`、`y.var(-1)` 验证，γ=1/β=0 时）。

完成判据：numpy 版打印出 `LN CHECK OK`（下一步对拍前先把形状 `(..., C)` 与 `atol` 定好）。

任务 2：LayerNorm / RMSNorm 与 torch 对拍（约 60 分钟）

新建 `week03/day3/norm_check.py`：

```
import numpy as np
import torch
import torch.nn.functional as Fn
rng = np.random.default_rng(0)                    # 统一 float64,seed 写死
x = rng.normal(size=(2, 5, 16)) * 1.7
g = 1 + rng.normal(size=16) * 0.1
b = rng.normal(size=16) * 0.1
def ln_np(z, gamma=None, beta=None, eps=1e-5):
    mu = z.mean(axis=-1, keepdims=True)
    var = z.var(axis=-1, keepdims=True)           # 有偏,和 torch 一致
    y = (z - mu) / np.sqrt(var + eps)
    if gamma is not None: y = y * gamma
    if beta is not None: y = y + beta
    return y
def rms_np(z, gamma=None, eps=1e-5):
    y = z / np.sqrt((z * z).mean(axis=-1, keepdims=True) + eps)
    return y if gamma is None else y * gamma
t = torch.tensor(x)
ref_ln = Fn.layer_norm(t, (16,), torch.tensor(g), torch.tensor(b), eps=1e-5).numpy()
mine_ln = ln_np(x, g, b)
print("LN  max|diff| =", np.abs(mine_ln - ref_ln).max())
assert np.abs(mine_ln - ref_ln).max() < 1e-12
# RMSNorm: torch>=2.4 有 torch.nn.RMSNorm;低版本用下面 3 行等价实现当参照
if hasattr(torch.nn, "RMSNorm"):
    m = torch.nn.RMSNorm(16, eps=1e-5).double()   # .double() 保证内部也在 float64 算
    m.weight.data = torch.tensor(g)
    with torch.no_grad():                         # 不加这句会在 .numpy() 处报 requires grad
        ref_rms = m(t).numpy()
else:
    ref_rms = (t / (t.pow(2).mean(-1, keepdim=True) + 1e-5).sqrt() * torch.tensor(g)).numpy()
mine_rms = rms_np(x, g)
print("RMS max|diff| =", np.abs(mine_rms - ref_rms).max())
assert np.abs(mine_rms - ref_rms).max() < 1e-12
# 两者的真实差异:RMSNorm 不减均值。用一个非零均值输入把它显出来
off = rng.normal(size=(2, 5, 16)) + 4.0
d = np.abs(ln_np(off) - rms_np(off)).max()
print("均值不为 0 时 LN 与 RMS 的最大偏离 =", d, "(均值=0 时应 <1e-12)")
print("NORM CROSSCHECK OK")
```

完成判据：打印 `NORM CROSSCHECK OK`，两个 diff 都 <1e-12，最后那行的偏离量级 **>1e-3**（说明 LN 减了均值而 RMS 没减——这就是它们唯一的实质差别，写进 NOTES，别背「RMSNorm 更简单更快」就完事）。

> 注：`torch.nn.RMSNorm` 是 2.4+ 才有的，脚本里已给降级分支。**别为了对拍去升级 torch**——把 GPU 环境搞坏的成本远大于这条注的价值。

任务 3：FFN、4C 扩展与 GELU / SwiGLU（约 75 分钟）

1. 写 `MLP(C, ffn_mult=4)`：`Linear(C, 4C) → GELU → Linear(4C, C)`，参数量 `8C²`；解释「先升维再降维」买到什么（每个 token 过一次非线性、把该头的输出与其他头的输出混合起来——**attention 负责混合 token，FFN 负责混合通道**，这句是面试可讲版）。

2. 手写 `gelu_np(x)` 两个版本：精确版（`0.5x(1+erf(x/√2))`，用 `math.erf` / `np` 实现）与 tanh 近似版（nanoGPT 的 `NewGELU`，系数 `0.044715`、`√(2/π)`）。对拍 `torch.nn.functional.gelu`（默认 `approximate='none'` = 精确版），报出两者与 torch 的最大偏离。**别背谁更准，跑出来。**
3. SwiGLU：结构是 `w1(C→d)`、`w2(C→d)`、`w3(d→C)`，前向 `w3( silu(w1(x)) * w2(x) )`——**三个矩阵而不是两个**。写一个形状单测证明你能把 `d` 取成 `8C/3` 附近时参数量与 `4C` 两矩阵版相当（面试考点：LLaMA 为什么把 FFN 中间维调小）。

完成判据：两个 GELU 版本与 `F.gelu` 的偏离各打印一次（精确版偏离应在 1e-12 量级，tanh 近似版在 1e-2 量级——**这是量级观察，不是实测值，你自己跑出来的数才算**）；SwiGLU 参数量等式在 NOTES 里成立（写出你选的 `d` 与实际 `numel()`）。

任务 4：组装 Block——Pre-LN vs Post-LN（约 110 分钟）

在 `week03/block.py` 里加 `class Block(nn.Module)`：

1. **Pre-LN（GPT-2/nanoGPT/LLaMA 的主流）**：`x = x + attn(ln_1(x))` → `x = x + ffn(ln_2(x))`；**Post-LN（原论文）**：`x = ln_1(x + attn(x))` → `x = ln_2(x + ffn(x))`。用一个 `mode="pre"/"post"` 开关实现，**别写两个类**。
2. 同一份参数、同一份输入，两种写法各前向一次，断言**输出形状都等于输入形状**（`(B,T,C)`），并把两者输出的差异打印出来。
3. 解释（写 NOTES，不许抄）：为什么 Pre-LN 训练更稳——残差主干是恒等映射，梯度能一路直传到最底层，且最坏情况下尺度只增不减；代价是需要在最后加一个 `ln_f`（nanoGPT/GPT-2 都有），而 Post-LN 每层出口已被归一化，不需要。
4. dropout 的取舍：Block 里 attn 后与 residual 后各一处，`p=0` 时前向必须与不写 dropout 完全一致（下面参数量对拍要用 `p=0`，理由记进 NOTES）。

完成判据：`python -c "from block import Block; import torch; b=Block(8,2,p=0.0); x=torch.randn(2,5,8,dtype=torch.float64); print(b(x).shape)"` 打印 `(2, 5, 8)`；pre/post 两模式都跑通；`print((pre_out-post_out).abs().max())` 的量级 > 1e-3（两者确实不同，不是笔误）；`week03/day1/test_block.py`（Day 1 那份）在改动后**依然全绿**。

> 注：改 Block 后必须回去重跑 Day 1 的 7 个单测。这一步很多人会跳，然后 Day 4 读 nanoGPT 时对不上形状，回头怀疑人生。**单测是防自己改坏的，不是给老师看的**——Week 6 改 LoRA 注入点、Week 9 改 rollout 逻辑时，这个习惯是唯一能救你的东西。

任务 5：参数量对拍——$12C^2$ 到底对不对 + 收工（约 45 分钟）

`print(sum(p.numel() for p in block.parameters()))`，与手算的 $12C^2$ + (线性层 bias 的零头) 对比（C=8 时很小，再用 C=256 手算一遍理论值 $12*256^2 = 786432$，然后构造一个 C=256 的 Block 实测）。

完成判据：两个 C 下实测值 = 手算值 + bias 零头，**误差恰好等于你数出来的 bias 个数**（差一个都不算过，把它当等式检查）。NOTES 写一句结论：**参数量 $\propto C^2$**，所以 `n_embd` 翻倍 → 参数 4 倍 → 显存与算力都 4 倍量级，这是 Week 5 调 config 的第一道硬约束。收工笔记：今天每次对拍失败的报错原文 + 修法各 ≥1 条进 NOTES。

今日验收（6.5h）：`NORM CROSSCHECK OK` + Block 形状不变 + pre/post 双模式 + 参数量对拍等式成立 + Day 1 单测重跑全绿。

Day 4 • 周四：逐行精读 nanoGPT `model.py` ——带审计表读，不许当小说看

今日总览（6.5h）：今天合上自己的代码，只读 nanoGPT。规则：**每条结论必须落到 `grep -n` 给你的行号上**，写在 `week03/day4/nanogpt_audit.md` 里（≥15 条带行号的断言）。这不是抄代码，是练一项后训练岗位的日常技能：**在一个陌生但极重要的仓库里，两小时内定位到你要改的那 20 行**。

任务 1：clone、目录导览与 grep 审计表开工（约 85 分钟）

```
cd E:/study/posttrain
git clone https://github.com/karpathy/nanoGPT
cd nanoGPT && ls && sed -n '1,40p' model.py
grep -n "class \|def " model.py
grep -n "n_embd\|n_head\|block_size" config/train_shakespeare_char.py
```

1. 先只回答三个问题：模型定义在哪个文件哪一段？训练循环在哪？config 怎么被读进来（`GPTConfig(**model_args)`）？
2. 建 `week03/day4/nanogpt_audit.md`，表头三列：`断言 | 证据（model.py:行号） | 我的疑问`；顺手记下 config 里 `n_layer / n_head / n_embd / block_size / learning_rate / max_iters / batch_size` 七个值（**以文件里实际写的为准，别背别人的数**），Day 5 上卡要用。

完成判据：审计表 ≥5 行带行号；七个 config 值抄进 NOTES，并注明来源文件名与行号。

> 注：读之前**先别搜「nanoGPT 逐行解读」的博客**。先看 30 分钟、自己写下 5 条疑问，再去看别人的讲解——顺序反过来你会以为自己在学，其实在读二手结论。这条规则贯穿 Week 4-9（LLaMA-Factory、TRL、verl 都是陌生仓库，能力可以迁移）。

任务 2：`CausalSelfAttention` 逐行（约 110 分钟）

审计表要覆盖下面每一条，逐条给行号（自己先猜再 grep）：

1. `c_attn = nn.Linear(n_embd, 3*n_embd)`：**一次算出 QKV**，与「三个独立矩阵」等价（Day 1 的 `test_head_split_equivalence` 就是你的理论依据）；好处是把三个 matmul 融成一个大的，kernel launch 更少。
2. `view / transpose` 的切头写法，与你 `block.py` 的是否一致。
3. `register_buffer("bias", tril...)`：为什么用 buffer 而不是普通属性（**它会随 `model.to(device)` 一起搬走，但不进 optimizer**）；写成 `-inf` 还是 `0.0`，为什么。
4. `block_size` 与 `T` 的关系：当 `T > block_size` 时 nanoGPT 怎么处理（`num_batches = ceil(T/block_size)` 再裁 buffer 的那段），以及为什么正常训练永远走不到这里（`block_size == 上下文长度`）。

5. `masked_fill` / `baddbmm`：新版用 `torch.baddbmm` 把 mask 融进 matmul，老版是 `att = att.masked_fill(bias==0, -inf)` ——**语义一致，写法不同**；grep `baddbmm` 看你在的版本。
6. `attn_probs` 为什么只在 `gradient_checkpointing=True` 时才保存（省显存的那段注释）。
7. dropout 的两处位置。

完成判据：审计表新增 ≥7 行带行号；`grep -n "baddbmm\|masked_fill\|register_buffer\|attn_probs" model.py` 四条全部命中并记录。

> 注：第 3 条那个 buffer 问题是 Week 4 的伏笔—— `register_buffer` 的东西**不会**出现在 `model.parameters()` 里，所以不进优化器；但 `state_dict()` 里有它，所以断点续训必须存 state_dict。你现在只要把这句话抄进 NOTES，Week 4 Day 2 做 checkpoint 时会突然懂。

任务 3：`MLP` 与 `Block` 逐行（约 85 分钟）

审计点：`c_fc(C→4C)`、`c_proj(4C→C)`；GELU 用的是 tanh 近似版（`NewGELU`，系数 0.044715）还是 `F.gelu`，grep 定位；`Block.forward` 里两行 `x = x + self.attn(self.ln_1(x))` / `x = x + self.mlp(self.ln_2(x))` ——**这就是你昨天写的 Pre-LN，找到行号，和你的实现逐字符比**；`LayerNorm` 用的是 nanoGPT 自定义的带 bias 版本还是 `nn.LayerNorm`（有 `FusedLayerNorm` 分支就注明）；残差支路权重初始化的 `scaled_init`：`nn.init.normal_(..., std=0.2/sqrt(2*n_layer))` 作用在哪些矩阵上（**只有投影层 `c_proj`**），为什么除以 `√(2·n_layer)`（一条 residual 路径上经过 attn 与 FFN 两个投影，累加 n_layer 层要保持方差守恒）。

完成判据：审计表 ≥6 行；NOTES 里用你自己的话解释 `0.2/√(2·n_layer)`，且能回答「为什么 `wte` 反而不 scaled 而 `c_proj` 要 scaled」。

> 注：`0.2` 这个数是 GPT-2 的 `init_range` 遗留，不是理论值；**理论上关键的是「按 residual 路径数缩放」这个结构**。面试问到 `0.2` 你别硬编一个不存在的推导，直说「经验值，重点是 √(2N) 这个因子」，比编一个更好。

任务 4：`GPT` 骨架与 weight tying（约 70 分钟）

审计点：`wte/vocab`、`wpe/block_size`、`h` ModuleList、`ln_f`、`lm_head`；`forward(idx, targets=None)` 里 `F.cross_entropy(logits.view(-1,V), targets.view(-1), ignore_index=-1)`（**`ignore_index=-1` 就是 Week 6 SFT loss masking 的机关**）；`configure_optimizers` 里 weight decay 只加在 `dim >= 2` 的参数上（为什么 embedding/norm 不加，写成一句 NOTES）；`from_pretrained` / `tie_weights`；`@torch.no_grad()` 的 `estimate_loss` 与 `generate` 的采样循环（`probs = F.softmax(logits, dim=-1)`、`topk`、`torch.multinomial`、温度 `logits/temperature` ——你 Week 2 学过的那套，在这里对上号）。

完成判据：审计表累计 ≥15 行且分布覆盖 4 个类（attention ≥5、MLP/Block ≥5、GPT 前向 ≥3、优化器/采样 ≥2）；NOTES 里有一句「weight tying 省下 `V×C` 参数、且让『看到某个 token』与『生成某个 token』共用一套表征」。

任务 5：细读笔记 30 条收口 + 收工（约 40 分钟）

1. 把今天 + 昨天的疑问与结论整理进 `week03/notes-nanogpt.md`，**凑满 30 条**，格式：`观察/疑问 → 证据(行号) → 结论`。少于 30 条不许进 Day 5（宁可从 Day 5 借时间，也别写 30 条口号）。

2. 对照 nanoGPT 的 `Block`，把你的 `block.py` 逐项列成**已一致 / 还缺**清单（缺的排到 Day 5 或 Week 4）。

完成判据：30 条笔记（带行号的 ≥20 条）；「已一致/还缺」清单 ≥8 行；审计表 + 笔记已 `git commit`（今天 commit 一次，别攒到 Day 6）。

今日验收（6.5h）：审计表 ≥15 行 + 30 条细读笔记 + 与 `block.py` 的差异清单。

Day 5 • 周五：差异清单、MHA/MQA/GQA/KV cache、与 SDPA 对拍、上卡 2 小时

今日总览（6h）：今天是本周唯一用卡的一天，也是把「读到的」变成「能讲的」的一天：① 写「nanoGPT 与论文原版的 ≥6 处差异」（每条带行号或实测依据）；② 讲清 attention 变体与 KV cache；③ 用 `F.scaled_dot_product_attention` 给你的手写 attention 做第三方对拍；④ 4090 上跑 1000 步 nanoGPT 字符级训练，用 forward hook 打印真实形状，和 Day 1 那张形状表逐项核对。

任务 1：与「Attention Is All You Need」的差异清单（约 70 分钟）

先翻论文（读架构那 3 页 + 缩放因子 √dk 那节即可，其余 Week 14 再细读），再写表。至少覆盖这几处，**每条要有依据（论文式子 / model.py 行号 / 你跑出来的数）**：

#	论文原版	nanoGPT	依据
1	encoder-decoder 双塔 + cross attention	只有 decoder 堆叠，单向因果掩码	
2	Post-LN：`LN(x+Sublayer(x))`	Pre-LN，并在最后加 `ln_f`	
3	FFN 激活 ReLU	GELU（tanh 近似版）	
4	正弦固定位置编码	可学习 `wpe`，与 `wte` 相加	
5	encoder/decoder 各一套 Wq/Wk/Wv	单塔，`c_attn` 一次算 3C	
6	输出 `softmax(W_o h)`，不共享	weight tying（`lm_head` 与 `wte` 同一张表）	
7	—	训练默认 fp32，AMP 是显式开关（LLaMA 类生产训练默认 bf16）	
8	—	norm 用 LayerNorm（LLaMA/GPT-NeoX 系用 RMSNorm）	

完成判据：表 ≥6 行且**每行的「依据」列非空**（行号或实测数字，写「我记得」视为未完成）；NOTES 补一句「哪些差异会影响训练稳定性、哪些只影响参数量」。

任务 2：MHA / MQA / GQA 与 KV cache（约 80 分钟）

1. 用 `hs = C/h` 这个符号，三种写清：MHA（h 套 Q/K/V）、MQA（1 套 K/V，h 套 Q）、GQA（g 组，`h/g` 个 Q head 共享一套 K/V）。
2. 手写 KV cache 的字节数公式：`2 × n_layer × C × T × bytes`（K 与 V 各一份，bf16 取 2）。用 nanoGPT 的实际 config（Day 4 抄的那七个值）算一个 batch=1、T=block_size 的 cache 大小，然后用 torch 实测验证：把同一 model 的 `wte` 输出与一段手写 cache 张量分配出来 `nbytes` 对比。
3. NOTES 写清两件事：**KV cache 为什么省算力**（自回归第 t 步时前 t−1 个 token 的 K/V 不变，重算每步 `O(T²)` → 命中缓存 `O(T)`）；**为什么 attention 复杂度是 `O(T²·C)`**（`att` 是 `(B,h,T,T)`，把 Day 1 打印过的形状引过来当证据）。加一句面试可讲版：GQA 用「几乎不掉点的质量」换「KV cache 与显存带宽除以 `h/g`」，这是 7B-70B 能长上下文服务的原因之一；MLA 只需知道「把 K/V 压成一个低秩 latent，进一步减 cache」，**本周不推公式**。

完成判据：cache 手算值与 `tensor.nbytes` 实测值**相对差 <1%**（对不上就是你把 `2×`、`bytes` 或 `C` 少乘了一项）；NOTES 有 KV cache 与 $O(T^2)$ 两条解释且各引用了一个你自己打印过的形状或数。

任务 3：与 `F.scaled_dot_product_attention` 第三方对拍（约 80 分钟）

新建 `week03/day5/sdpa_check.py`（float64，seed 写死）：

```
import torch
import torch.nn.functional as F
from block import causal_multihead_attention
torch.manual_seed(11)
B, T, C, H = 2, 9, 16, 4
x = torch.randn(B, T, C, dtype=torch.float64)
Wq, Wk, Wv, Wo = (torch.randn(C, C, dtype=torch.float64) * 0.5 for _ in range(4))
hs = C // H
def sdpa_version(x):
    q = (x @ Wq).view(B, T, H, hs).transpose(1, 2)     # (B,H,T,hs)
    k = (x @ Wk).view(B, T, H, hs).transpose(1, 2)
    v = (x @ Wv).view(B, T, H, hs).transpose(1, 2)
    y = F.scaled_dot_product_attention(q, k, v, is_causal=True)   # 内部已做 1/sqrt(hs) 与因果掩码
    return y.transpose(1, 2).reshape(B, T, C) @ Wo
mine, _ = causal_multihead_attention(x, Wq, Wk, Wv, Wo, H)
ref = sdpa_version(x)
d = (mine - ref).abs().max().item()
print("max|mine - sdpa| =", d)
assert d < 1e-12, "和官方实现不一致,先查缩放用了 hs 还是 C、以及 T 是否等于 block_size"
print("SDPA CROSSCHECK OK")
```

完成判据：打印 `SDPA CROSSCHECK OK`（diff <1e-12）；NOTES 写一条「`is_causal=True` 的语义是**前缀掩码**，所以要求 `T ≤` 你给定的上下文长度；当 `T` 小于缓存长度时不能图省事用它，要传显式 `attention_mask`」——这句在 Week 9 做 GRPO rollout 时真的会撞上。

任务 4：上卡——1000 步真实训练 + forward hook 形状验证 + 归档（约 130 分

钟)

1. 开 AutoDL 4090（**开卡前**先把下面命令写进 `week03/day5/run.sh`，跑完立刻关机）：

```
# 实例内,内容全部落 /root/autodl-tmp 以免镜像重置丢失
source /etc/network_turbo
git clone https://github.com/karpathy/nanoGPT /root/autodl-tmp/nanoGPT && cd /root/autodl-tmp/nanoGPT
pip install transformers datasets tqdm -i https://pypi.tuna.tsinghua.edu.cn/simple
python data/shakespeare_char/prepare.py
python train.py --dataset shakespeare_char --max_iters 1000 --eval_interval 200 --out_dir /root/autodl-tmp/out-w03
nvidia-smi
```

记下打印出来的 `vocab_size`、迭代速度、`train loss` 与 `val loss` 的起止值（**PPL 用上周公式 `exp(loss)` 换算，和「随机基线 = vocab_size」比一次**）。

2. 写一个 hook（本地或实例上都行，5 行）验证形状表：

```
import torch
from model import GPT, GPTConfig   # 在 nanoGPT 仓库里跑,或把 Block 拷出来
cfg = GPTConfig(block_size=64, vocab_size=65, n_layer=2, n_head=4, n_embd=32, dropout=0.0)
m = GPT(cfg).double()
shapes = {}
def hook(name):
    def fn(mod, i, o):
        shapes[name] = tuple(i[0].shape) if isinstance(i, tuple) else tuple(i.shape)
    return fn
for n, mod in m.named_modules():
    mod.register_forward_hook(hook(n))
m(torch.randint(0, 65, (2, 8)))
for k, v in shapes.items(): print(k, v)
```

3. 归档：把「前向形状表」（Day 1 起的那张，今天补全）、loss 曲线截图、`SDPA CROSSCHECK OK` 截图拷进 `llm-posttraining-lab/blogs/drafts/week03-materials/`，标注「进 Week 5《我从零训了一个 GPT》的『我的 Block 与 nanoGPT 的差距』一节」。**本周不发博客。**

完成判据：`train.py` 跑完 1000 步，`val loss` 明显低于第 0 步的 `ln(vocab_size)`（字符级 shakespeare 的 vocab 约 65，`ln 65 ≈ 4.17`，**以你 print 出的那个数为准**；**用 `exp(val_loss) / vocab_size` 这个比值来判断有没有超过随机基线，<1 才算学到东西**）；hook 输出的 `transformer.h.0.attn` 输入形状 = `(2, 8, 32)`，与你 Day 1 形状表一致；1000 步那四个数（起止 loss + it/s + 显存峰值）写进 NOTES；drafts 目录 ≥3 个文件 + 一行归属说明；卡已关机（控制台显示「已关机」）。没抢到 4090 → **换任意有货的卡**（A100/A10/G 系列都行，这个配置极小）。

> 注：这一步的显存占用应该在**几百 MB 到 2 GB** 量级（模型只有几万参数，激活主导）。如果你看到 10 GB+，大概率是 `nvidia-smi` 里连显存分配器缓存 + 别的进程一起看了——这正好是 Week 4 要讲的「`memory_allocated` vs `memory_reserved` 」区别，先记进 gaps。

今日验收（6h）：差异表 ≥6 行有据 + cache 手算/实测 <1% + `SDPA CROSSCHECK OK` + 1000 步 loss 低于随机基线。

Day 6 • 周六：输出日——白板默写、形状表定稿、闭卷自测、repo 收口

今日总览（6h）：今天不学新东西，把本周的东西**变成能交付的表达**：默写 Block、定稿形状表（这张表是你 Week 5 排查形状问题、Week 13 手撕题的底子）、做 4 题闭卷自测（含 Week 1 推来的那道第 4 题），然后整理 push。

任务 1：白板默写 decoder Block（约 45 分钟）

合上所有代码，纸上画 **Pre-LN decoder Block**：标出 `ln_1 → attn → 残差` 与 `ln_2 → FFN → 残差` 两条子层，写明每个框的输入输出形状，并在图旁写 `ln_f` 为什么存在。然后**再画一张 Post-LN 版**，两图并排。

完成判据：两张图各 ≤5 分钟画完（计时）；对照 `block.py` 自查，错的地方用红笔改并写一句「我为什么错」。**画图不许参照任何文件**——参照了就等于没测。

任务 2：前向形状表定稿（约 55 分钟）

`week03/day6/shapes.md`：一张 **12 行以上**的表，从 `idx (B,T)` 到 `logits (B,T,V)`，包含 `wte/wpe/x`、`ln_1(x)`、`q/k/v`（含头的形状）、`att`、`y`（合并头后）、`residual 相加后`、`ln_2`、FFN 中间维、`logits`。每列：`变量 | 形状 | 关键操作 | 我在哪个文件实测过`。

完成判据：≥12 行且「实测过」一列**每行都有来源**（`block.py` / nanoGPT `model.py:行号` / hook 输出）；FFN 中间维写成 `4C`；`att` 写成 `(B,h,T,T)` 并标一句 `O(T²)` 显存来源。

任务 3：闭卷自测 4 题与订正（约 120 分钟，计时 60 分钟做题）

1. 画出 Transformer decoder Block 结构图（Pre-LN 与 Post-LN 两种都画，并说清 LN 在残差加之前还是之后、GPT-2/nanoGPT 用哪种、为什么主流选它）——**这就是 Week 1 自测第 4 题，今天正式交卷。**
2. 写出带 batch 的多头因果注意力完整前向（伪代码，`x:(B,T,C)` 进、`(B,T,C)` 出，标注每步形状与缩放用的是 `hs` 还是 `C`）。
3. 说明 `wte` 为什么要乘 `√n_embd`、LayerNorm 与 RMSNorm 的实质差别（一句话，不许写"更简单"）、以及为什么 LLaMA 系选 RMSNorm。
4. KV cache 里存的是什么、为什么它省算力、MQA/GQA 怎么减它、RoPE 为什么不需要 `wpe` 表。

计时 60 分钟做完四题再往下看参考答案（自己判，宽松但要有依据）：

- 第 1 题：Pre-LN `x = x + attn(ln(x))`，Post-LN `x = ln(x + attn(x))`；Pre-LN 残差主干是恒等、梯度直通、深层更稳，代价是出口需 `ln_f`。
- 第 2 题：`q,k,v = split(x@W_)` → `(B,h,T,hs)`；`att = softmax(qkᵀ/√hs + causal_mask) @ v` → merge → `@Wo`；缩放用 `√hs`；mask 在 softmax 之前。
- 第 3 题：`√n_embd` 是为了让 token 身份在与 `wpe` 相加、被 LN 归一化后仍占主导；LN 减均值除标准差 + γ/β，RMSNorm 只除以 `√(mean(x²))` + γ（**不减均值**），少一次归约、更省；LLaMA 用 RMSNorm 主要图

快与够用（GPT-2 时代用 LN 是历史与 TF 实现一致）。

- 第 4 题：cache 存历史 token 的 K/V；避免每步重算前缀；GQA 让 g 组共享 K/V → cache 除以 `h/g`；RoPE 把位置编码进 q/k 的旋转里，无需查表。

完成判据：四题都写了（空着算失败）、计时 ≤60 分钟；每题有「对/半对/错」标记 + 错的订正痕迹；至少 1 条订正转化为 gaps 条目并给归属周。第 1、2 题是本期硬指标——**写不出就今晚重做对应任务**（第 1 题→任务 1 重画，第 2 题→Day 1 任务 3 重默），不许推到 Week 4。

任务 5：repo 收口与 push（约 95 分钟）

1. `E:/study/posttrain/week03/` 整理成可独立运行的模块：`block.py`（attention + Block，pre/post 开关）、`test_block.py`（Day 1 的 7 测 + Day 2 的等变性）、`norm_check.py`、`rope.py`、`sdpa_check.py`、`nanogpt_audit.md`、`shapes.md`、`README.md`。
2. README 四段：是什么 / 怎么跑测试（`PYTHONPATH=. python day1/test_block.py`、`python norm_check.py`）/ 关键结论（形状表、差异表、参数量 `12C²`）/ **哪些跟敲 Karpathy、哪些自己写**（诚信声明）。同步拷进 `llm-posttraining-lab/notes/week03-block/` 并 push。

完成判据：在新位置 `python test_block.py` 打印 `ALL BLOCK TESTS PASS`、`python sdpa_check.py` 打印 `SDPA CROSSCHECK OK`；GitHub 链接可访问且 README 渲染正常；链接记进 NOTES 顶部。

任务 6：gaps 结算与新洞映射 + 周复盘（约 45 分钟）

1. 更新 `week03/week03-gaps.md`（≥6 条，每条有「哪天/哪周 + 动作」），新洞必须落进下面这张表的具体格子：

本周暴露的洞	归到	为什么那天补上
`block.py` 还没和 nanoGPT 完全对齐（初始化 std、dropout、tying、`ln_f`）	**Week 4 Day 2 + Day 6**	那周要把它塞进完整训练循环跑通，不对齐就会当场发现
BatchNorm 与 `model.train()/eval()` 的行为差别	**Week 4 Day 2**	训练循环模板里 dropout/BN 是唯一的行为开关
混合精度、梯度累积、checkpoint、lr 调度器	**Week 4 Day 3/4/5**	那周主线就是「会跑训练」
Pre-LN vs Post-LN 的稳定性差别我只讲不练	**Week 4 Day 6 + Week 5**	上卡跑 1000 步时同配置改一处，看 loss 曲线是否发散
真实分词（BPE）与 `block_size` 的实际影响、采样参数对照	**Week 5**	M1 交付里有 BPE 数据准备与 temperature/top-k 小表
attention mask 在推理期（cache 场景）怎么写、KV cache 我只算了没实现	**Week 5 采样 + Week 9 rollout**	GRPO 的 rollout 会逼你面对 cache 与 `is_causal` 的坑
RoPE 外推（NTK/YaRN）、MLA、MoE 只到「能讲名词」	**Week 12 评测 + Week 14 八股第二批**	面试八股集中期补齐

2. `week03/review.md` 五块（产出清单 / 第 4 题现在的掌握度并对比 Week 1 答卷 / 按天真实时间 vs 42h / 最有价值的一件事 / 下周最想补的洞）。

完成判据：`week03-gaps.md` ≥6 条且**每条有周次**；review.md 第 ② 块写了「Week 1 那次这题答不出，今天能答到什么程度」的对照。

今日验收（6h）：两张默写图 + 形状表 ≥12 行 + 自测 4 题有判分 + repo push 成功 + gaps 有归属。

Day 7 • 周日：缓冲补漏 + 闭卷复测 + 总验收 + Week 4 预习

今日总览（4.5h）：今天不排新内容。补漏最多 2 个洞、闭卷复测 3 题、做总验收、30 分钟预习下周。**下周是这套计划第一次认真用卡的一周**，今晚把作息调回来。

任务 1：补漏（约 95 分钟）

打开 `week03-gaps.md` 与 `review.md` 第 ② 块，**只补 2 个洞**，优先「Block 与 nanoGPT 未对齐」和「形状表里对不上号的那一行」。补法：重跑对应脚本 + 改 `block.py`，不是重读笔记。

完成判据：两条在 gaps 里变 `[x]`，各附一句「怎么补的 + 复跑打印了什么」；`block.py` 改动后 Day 1 的 7 测仍全绿。

任务 2：闭卷复测 3 题与订正（约 75 分钟）

合上一切，重做：① 默写 Pre-LN Block 并解释残差主干与 `ln_f`；② 写多头因果注意力前向（形状注释齐全）；③ 三句话分别解释 LN vs RMSNorm、weight tying、KV cache 各买到什么。

订正：对照 `block.py`、`shapes.md` 与 Day 6 批改，订正写在原卷上。**如果 ② 又写错，把 Week 4 Day 1 的前 60 分钟改成重做 Day 1 任务 3**（这件事比 Week 4 进度重要，attention 写不出后面什么都白搭）。

完成判据：①② 形状与顺序全对（对照 Day 6 的答卷，**这次不看答案**）；③ 每句 ≤2 行且不含「更简单/更快」这类没有理由的词；订正痕迹在原卷上可见，结果与是否需要占用 Week 4 Day 1 都记进 `review.md` / `week03-gaps.md` 顶部。

任务 3：总验收（约 30 分钟）

#	验收项	判据
1	多头因果注意力	`block.py` 两种实现（transpose 版 + einsum 版）一致，`ATTENTION BUILT OK`
2	单测	7 个测试 PASS，含 `test_causal_output_unchanged` 与头切分等价测试
3	位置编码	置换等变实验 True/False 两行符合预期；`ROPE RELATIVE OK`；位置对照表 5 行
4	norm 与 FFN	`NORM CROSSCHECK OK`（LN/RMS diff <1e-12，且 LN≠RMS 的偏离 >1e-3）；GELU 两版偏离已打印；SwiGLU 参数量等式成立

#	验收项	判据
5	Block 与参数量	pre/post 双模式输出形状 = 输入形状；`numel()` 对上 `12C² + bias`
6	nanoGPT 精读	审计表 ≥15 条带行号；差异表 ≥6 行有据；细读笔记 30 条
7	对拍与实跑	`SDPA CROSSCHECK OK`（diff <1e-12）；KV cache 手算/实测 <1%；4090 上 1000 步 val loss 优于随机基线（`exp(val_loss)/vocab_size < 1`）；形状表 ≥12 行且每行有实测来源；repo 已 push

≥6 项通过 = 本周合格；第 1/2/4 项不许缺——它们是 Week 4 训练循环和 Week 5 调 config 的直接前置。

完成判据：验收表 7 行都有状态与一行原因，抄进 `review.md` 末尾。

任务 4：下周预习与收工（约 70 分钟）

1. 翻 nanoGPT `train.py` 30 分钟（只看不跑），回答三件事：`get_batch` 在哪里、`forward_backward_micro_step` 或 `lossf` 那层的梯度累积怎么写的、checkpoint 存了哪几样（`model` / `optimizer` / `iter_num` / `rng_state`）。
2. 把「我下周四件事的主线」写进 NOTES：**Week 4 = 把 `block.py` 塞进一个我能自己写的训练循环里，并且看得懂每一行在干什么**（Dataset/DataLoader、checkpoint 续训、手写 AdamW 对拍、bf16/累积/裁剪/checkpointing、显存解剖）。
3. review.md 末尾写一行本周位置：micrograd → makemore（会定义 loss）→ **nanoGPT Block（知道 loss 在优化什么结构）** → PyTorch 训练系统（会跑训练）→ M1（从零训一个 GPT）→ 之后才是后训练。
4. 收工：本周产物 `git status` 检查一遍（未 commit 的补齐），确认 gaps 每条都有周次归属。**Week 4 周四晚要预约卡**（Day 5/Day 6 用卡），看一眼 AutoDL 余额（不够就充 ¥100，周 2-5 全程卡时预算 30h ≈ ¥45-75）。

完成判据：三个问题各写 ≥2 行且带 `train.py:行号`；review.md 有那一行位置说明；`git status` 干净、卡余额已确认。

今日验收（4.5h）：验收表 ≥6 项 + 复测 ①② 全对 + 预习三问有行号 + 卡余额确认。

三、常见问题与纪律

- **`view` / `reshape` / `transpose` 一直报错怎么办？** 顺序：① 打印 `.shape` 与 `.is_contiguous()` ② transpose 后一律 `reshape`（它会拷贝）③ 别在 `nn.Parameter` 上 reshape（会得到没意义的梯度路径）。记住一句话：**先打印形状，再推理，永远。**
- **对拍失败（SDPA / norm / 头切分）的排查顺序**：① dtype 是否两边都 float64 ② 缩放用了 `hs` 还是 `C` ③ mask 对角线（`diagonal=1` 才对）④ softmax 的 `dim` ⑤ 头的切分与合并是否对称（`view(...,H,hs).transpose` 与 `transpose(...).reshape` 必须成对）⑥ dropout 是否两边不同（`p=0` 或统一 `eval()`）。**把 diff 打印出来再 assert**，只看 `AssertionError` 是浪费时间。

- **读源码读困了**：切换到「带问题 grep」模式（每 10 分钟必须落一条行号结论），或者站起来用嘴讲「这个 forward 的输入输出是什么形状」。**不要靠二刷视频代替读码**——Week 6 起的 LLaMA-Factory / TRL / verl 全是只能读码的仓库。
- **卡时超了 / 卡抢不到**：本周上限 2h。Day 5 那 1000 步若 40 分钟还没跑完，就是配置或环境有问题（这个模型只有几万参数），直接 Ctrl+C、**先关机**，现场记进 gaps，别烧钱等结果；4090 无货就换任意型号（这个配置对卡不敏感），**别刷页面等卡**，实在开不了就把 hook 部分留本地 CPU 跑，loss 曲线推到 Week 5 一起做并在 gaps 写明。
- **某天没完成**：不熬夜追。按「天-任务号」写进 `week03-gaps.md`，Day 6 缓冲与 Day 7 是留给它们的。连续两天崩就砍 Day 2 任务 5 的对照表与 Day 5 任务 2 的 MLA 部分，**永远优先保 Day 1 的单测、Day 3 的 Block、Day 6 的默写与自测**。
- **AI 工具边界**：允许解释报错、讲概念、review 你写好的文件；**禁止**让它生成 `block.py` / `rope.py` / `norm_check.py` ——这三个文件是本周的肌肉记忆。允许在**默写与自测完成之后**用 AI 逐行讲 nanoGPT，但要先把审计表填完。
- **诚信与合规**：README 里写清哪些跟敲 Karpathy、哪些自己写；本周用的莎士比亚文本与 nanoGPT 的数据脚本都是公开材料，**Week 7 起涉及真实产品数据，脱敏规则开始生效**。